

Q1. Sorting: Please analyze the given array of data and determine which sorting algorithm would be the best choice. [2+ 2 + 2 + 2 = **8** Marks]

1. Array [1, 2, 3, 5, 4, 6]

- a. **Insertion**
- b. Bubble
- c. Selection

2. Array [2, 4, 5, 3, 6, 7]

- a. **Insertion**
- b. Bubble
- c. Selection

3. Array [5, 6, 1, 2, 3, 4]

- a. Insertion
- b. **Bubble**
- c. Selection

4. Array [4, 9, 11, 8, 3, 6]

- a. Insertion
- b. **Bubble**
- c. **Selection**

Q2. You are provided with the following function, which has formal parameters for an **array** and its size (**int arr[], int n**). Please answer the questions below the code. [5 + 3 = **8** Marks]

```
void Chentapdumdum(int arr[], int n)
{
    for (int i = n / 2 - 1; i >= 0; i--)
    {
        int j = i;
        while (true)
        {
            int left = 2 * j + 1;
            int right = 2 * j + 2;
            int smallest = j;

            if (left < n && arr[left] < arr[smallest])
                smallest = left;
            if (right < n && arr[right] < arr[smallest])
                smallest = right;

            if (smallest != j)
            {
                swap(arr[j], arr[smallest]);
                j = smallest;
            }
            else
            {
                break;
            }
        }
    }
}
```

1. If the values of arrays pass to the function are { 10, 20, 29, 22, 14, 51, 16}, what will be the resultant array after function call?

Answer:

0	1	2	3	4	5	6
10	14	16	22	20	51	29

2. What will be the time complexity of above code?

Lower Bound $O(n)$ and Upper Bound $O(n\log(n))$

Q3. What will be the Time Complexity of Codes in term of Big - Oh?

[5+5 = **10 Marks**]

Code	Time Complexity
<pre>void Time_Bomb(int n) { int count = 0; for (int i = 1; i <= n; i *= 2) for (int j = 0; j < n; j++) for (int k = 0; k < j; k++) count++; }</pre>	Overall Time Complexity: $O(n^2 \log(n))$
<pre>Void Game(int n) { int count=0; for(int i=1; i*i<n ; i++) for(int j=1; j<=i ; j++) count++; }</pre>	Overall Time Complexity: $O(n)$

Q4. You are given a double linked list with 8 nodes containing values sequentially from 1 to 8. The head node points to the first element with value 1. The list is circular, so the **next** pointer of the last node (with value 8) points back to the first node, and the **prev** pointer of the first node points to the last node. **Head → 1 ↔ 2 ↔ 3 ↔ 4 ↔ 5 ↔ 6 ↔ 7 ↔ 8**

[6+4+4 = **14 Marks**]

```
void List :: Nodinator()
{
    Node* ptr = head;
    for (int i = 1; ptr->next != ptr; i = i * 2)
    {
        for (int j = 1; j <= i; j++)
            ptr = ptr->next;
        if (head == ptr)
            head = head->next;
        cout << ptr->data << "\t";
        ptr->prev->next = ptr->next;
        ptr->next->prev = ptr->prev;
        Node* temp = ptr;
        ptr = ptr->next;
        delete temp;
    }
}
```

1. What will be the output when the head node is passed to the **Nodinator** function?

1	2	3	4	5	6	7
2	5	3	8	1	7	4

2. Draw the linked list after the function call.

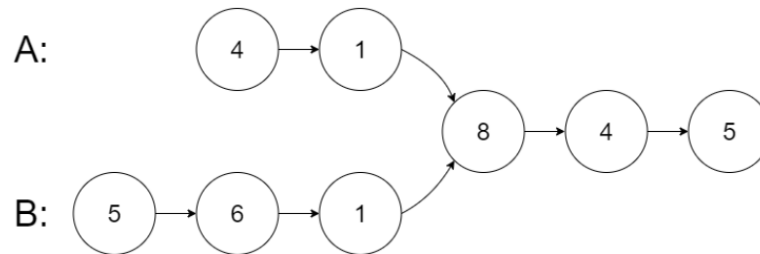
Only 6th node will be left

3. What is the time complexity of the given function in Big-O terms?

$O(2^n)$

Q5. Given the heads of two singly linked-lists headA and headB, return the node at which the two lists intersect. If the two linked lists have no intersection at all, return null. **[15 Marks]**

For example, the following two linked lists begin to intersect at node 8:



Output: Intersection Node is 8

```
Node *getIntersectionNode(Node *headA, Node *headB) {
    Node *ptrA = headA;
    Node *ptrB = headB;
    if (ptrA == NULL || ptrB == NULL )
        return nullptr;

    while (ptrA != ptrB) {

ptrA = ptrA != NULL ? ptrA->next : headB;

ptrB = ptrB != NULL ? ptrB->next : headA;
    } return ptrA;
}
```

Q6. You've been hired by the government for a critical mission. As the world's top programmer, your task is to infiltrate the enemy's most secure server, extract critical information regarding agent identities, and delete it—leaving no trace—all in constant time.

You bypass all defenses and reach the core. The target is in sight: a pointer to highly classified data.

The data is stored in a **singly linked list**, but there's a catch.

You don't have access to the head of the list. The only thing you've been given is a pointer directly to the node containing the critical information. In addition, you also know that agent information is in middle of the link list (pointer is neither at the beginning nor the end). Your mission is clear: extract the information and delete it from the list without leaving any trace. No backtracking, no full traversal of the list. The AI watchdogs would detect you if you attempted to traverse through the server's memory. You need to complete the task in $O(1)$ time. You can't afford to fail 😊.

[15 Marks]

Example: current pointer pointing to Rizwan's Node.

... [Babar Azam] → [Shadab Khan] → [Rizwan] → [Fakhar Zaman] → [Naseem Shah] → ...

After deleting/removing Rizwan

... [Babar Azam] → [Shadab Khan] → [Fakhar Zaman] → [Naseem Shah] → ...

```
// structure to Node is
struct Node{
    string agentName;
    Node * next;
};

string deletingCurrent(Node * current){
    string name = current -> agentName;

    Node *ptr = current->next;
    current -> data = ptr -> data;
    current -> next = ptr -> next;
    delete ptr;
    return name;
}
```